19. # Distributed Cache and Caching Strategies

Lecture. 56

- What is Caching?
- What is Distributed Caching and how its done?
- Caching Strategy:
  - Cache aside
  - Read-Through
  - Write Around
  - Write through
  - Write Back
- Cache Eviction Policies
  - LRU
  - FIFO
  - LFU etc.

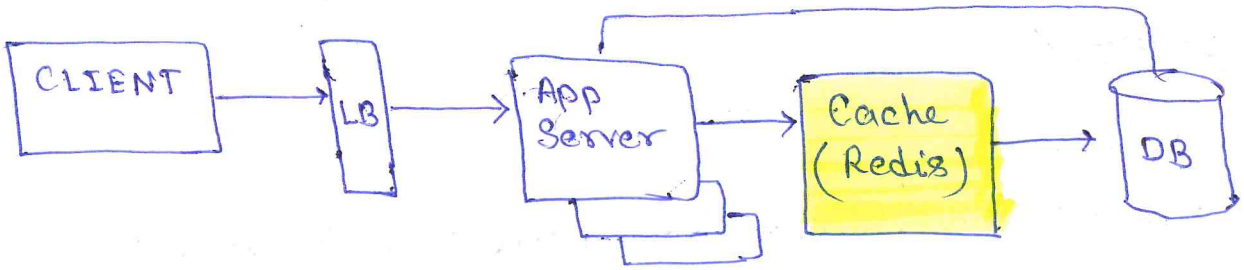
Concept && Coding

What is Caching?

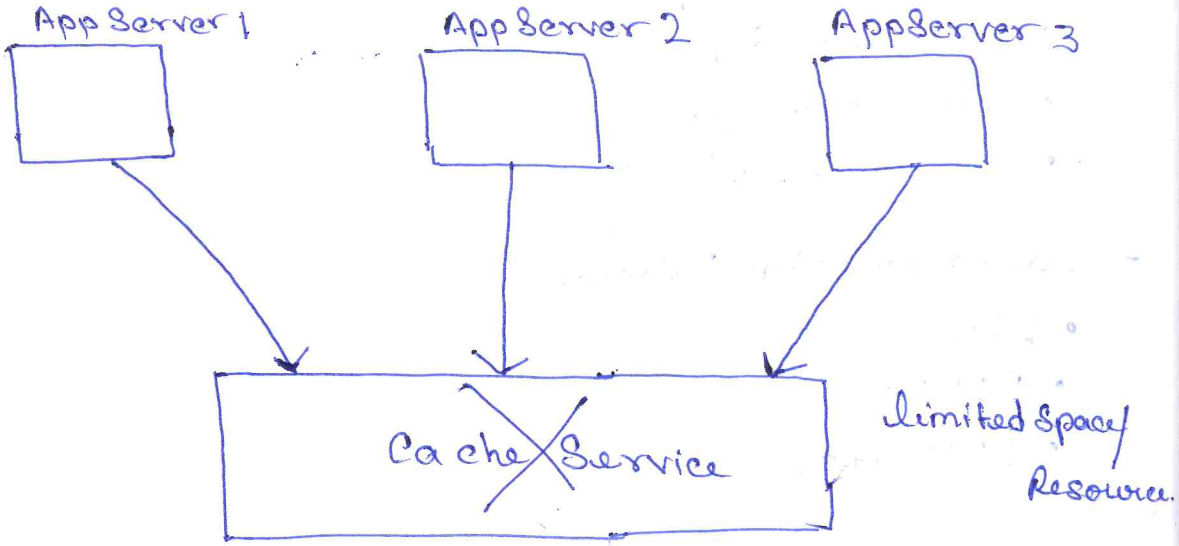
- Caching is a technique to store frequently used data in a fast access memory rather than acessing data every time from slow access memory.
- This makes our system very fast.
- It helps to reduce the Latency.
- It also helps to achieve the fault tolerance.

There are different types of caching present at different layer of system like:

- Client sid Caching (Browser Caching)
- CDN (used to store the static data).
- Load Balancer
- Server Side Application Caching (like Redis etc).



What is Distributed Caching?

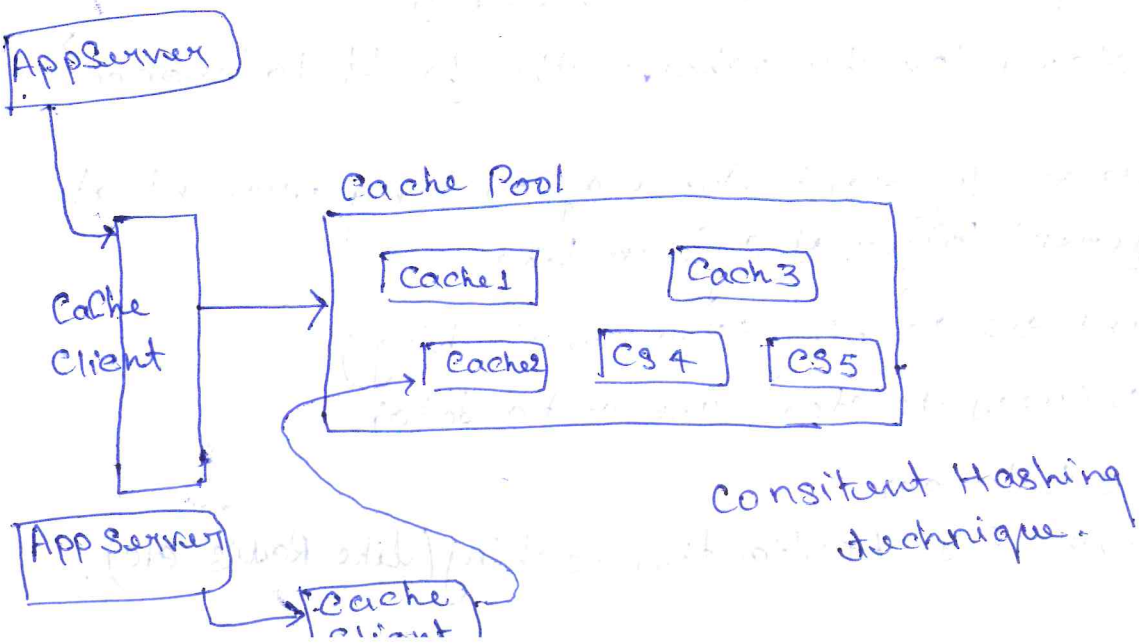


Concept && Coding

Limitation

1. Scalability
2. Single point of failure

That resion Distributed Caching in picture.

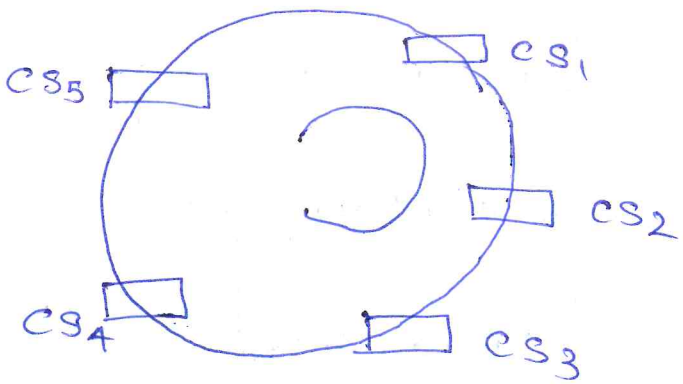


Consitent Hashing technique.

* App Server has own cache client and using that cache client it will get one of the cache server where it will store the data or read the data.

How one of the Server would be alloted?

For that it uses Consistent Hashing technique.



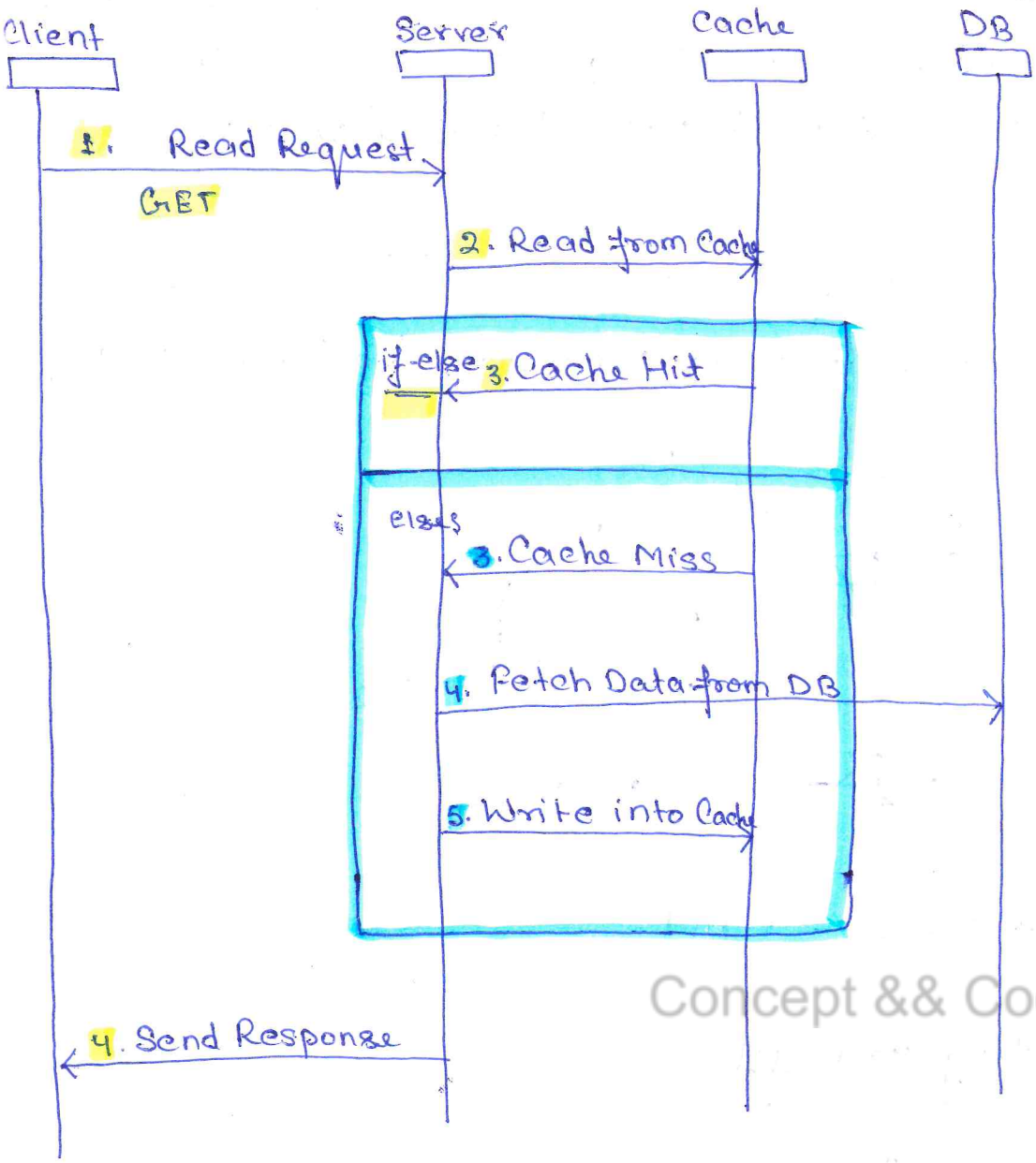
App Serv 1 ⟶ put

Concept && Coding

Caching Strategy :-

1. Cache Aside :-

- Application first check the cache.
- If data found in cache, it's called Cache Hit and data is returned to the client.
- If Data is not found in Cache, it called Cache Miss. Application fetch the data from DB, store it back to Cache and data is returned to the client.



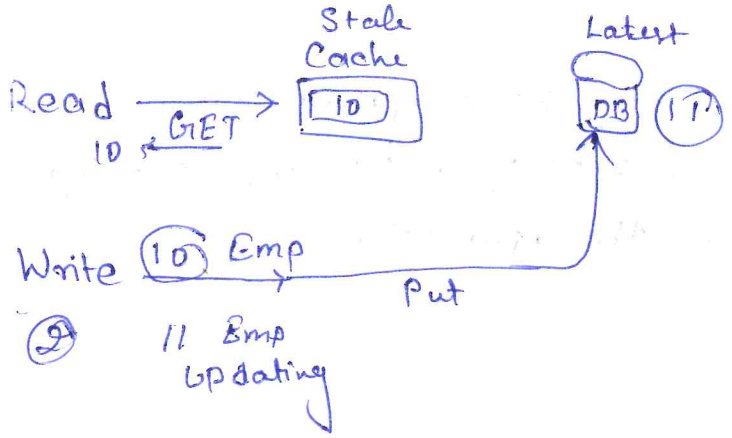
Concept && Coding

PROS:

- Good approach for Heavy Read Application:
- Even cache is down, request will not fail, as it will fetch the data from DB
- Cache Document data structure can different than the Data present in DB.

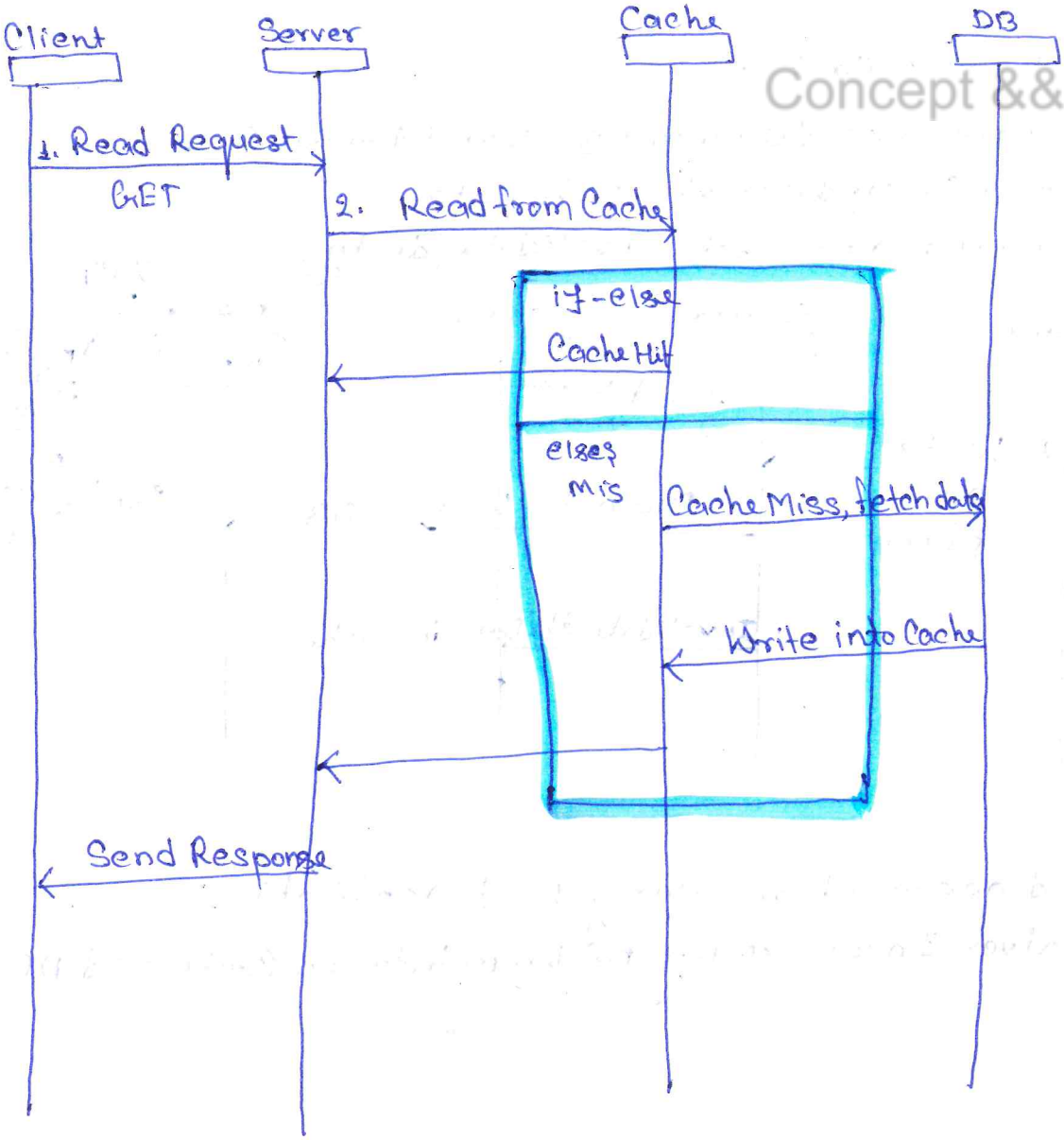
CONS:-

- For new data read, there will always be CACH-MISS first (to resolve this, generally we can pre-heat the cache)
- Without approprite caching is not during Write operation. There is chance of Inconsistency between cache and DB.



2. Read Through Cache :

- Application first check the cache.
- If data found in Cache, It's called Cache Hit and data is returned to the client.
- If Data is not found in Cache, its Called Cache Miss. Cache Library itself fetch the data from DB, store it back to Cache and data is returned to application.



Concept && Coding

PROS

- Good approach for Heavy Read application.
- Logic of fetching the data from DB and updating Cache is separated from the application.

CONS :-

- For new data read, there will always be CACHE-MISS first (to resolve this, generally we can pre-heat the cache)

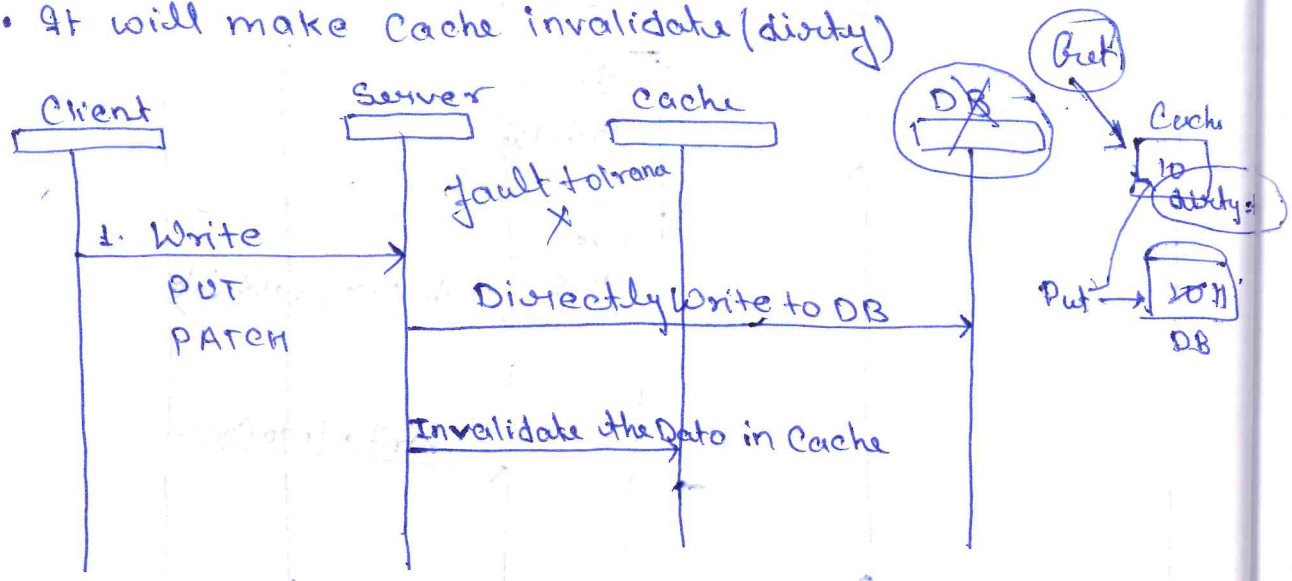
- Without appropiate caching is not used during Write operation. There is chance of inconsistency between Cache and DB.

- Cache Document structure should same as DB table.

3. Write Around Cache :-

Concept && Coding

- Directly writes data into DB. Don't touches Cache.
- It do not updates the cache data.
- It will make Cache invalidate (dirty)



PROS

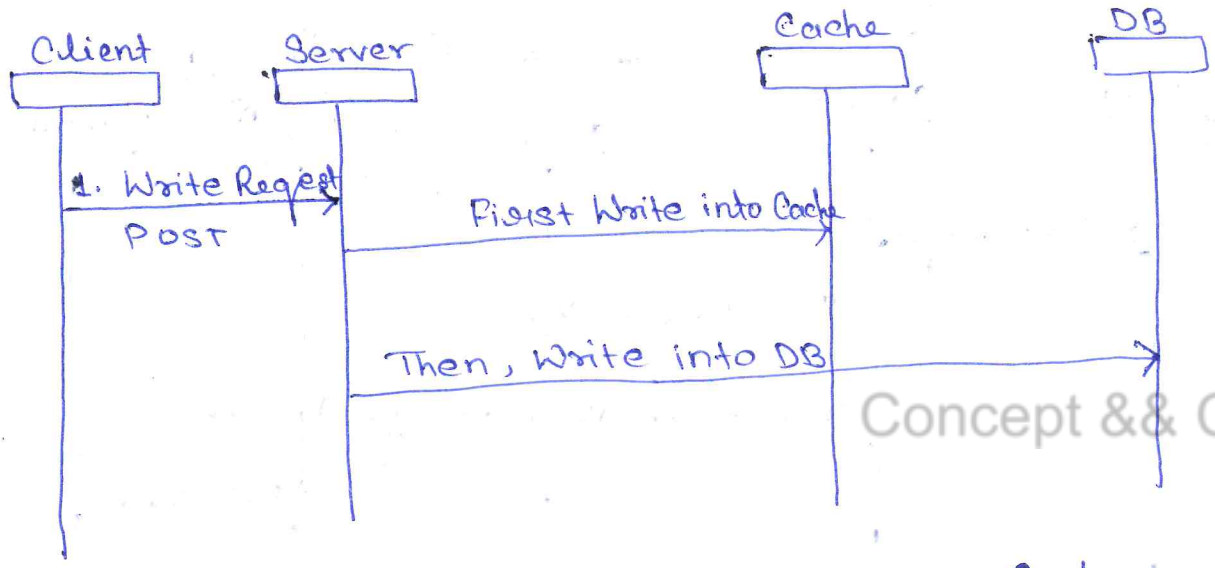
- Good approach for Heavy Read application.
- Resolves Inconsistency problem between Cache and DB.

Cons :-

- For new data read, there will always be CACHE-MISS (to resolve this, generally we can pre-heat the cache)
- If DB is down Write operation will Fail.

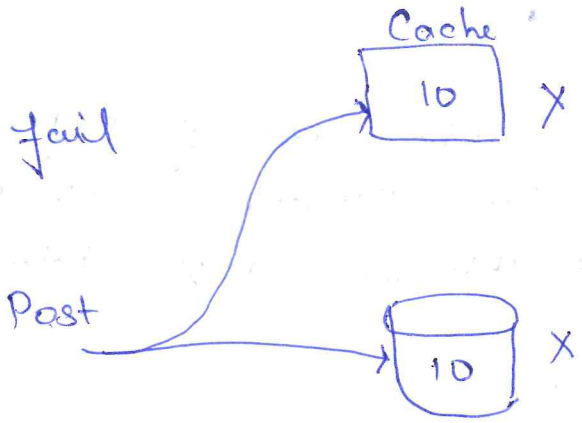
4. Write Through Cache :-

- First Writes data into Cache.
- Then in synchronous Writes data into DB.



Concept && Coding

* If any of insertion will fail throw the exception.



PROS

- Cache and DB always remain consistent.
- Cache Hit chance increases a lot.

CONS :-

- Alone its not useful, it will increase the latency. (that's why its always used with Read through or aside cache)

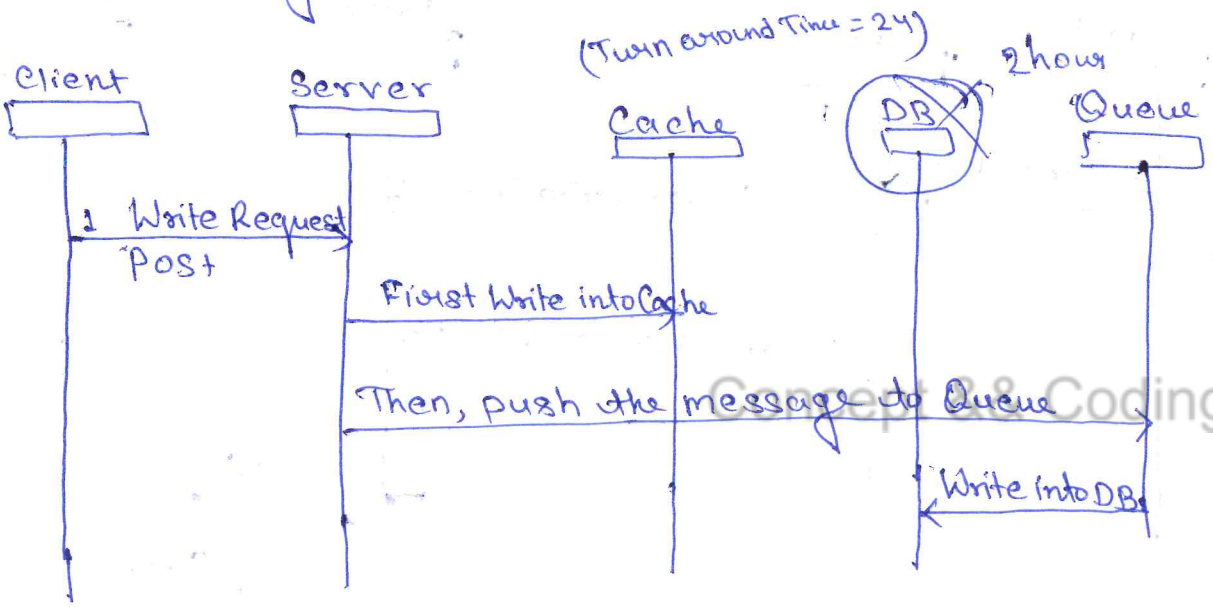
~~- 2 Phase~~

- 2 Phase commit, need to be supported with this. To maintain the transactional property.
- If DB is down write operation will fail.

It still not fully fault tolerance

5. Write Back (or Behind) Cache :-

- First Writes data into the Cache
- Then in Asynchronous Write data into the DB



Concept & Coding

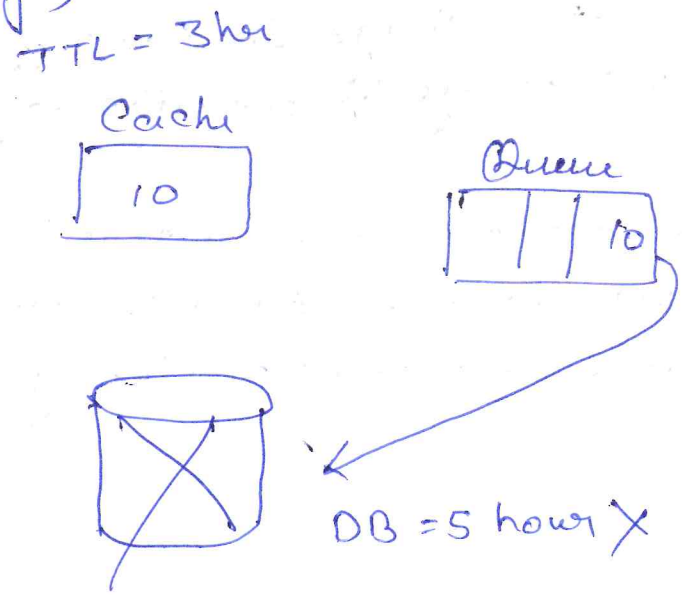
* 24 hour due full fill the data on cache.
* When DB come back Queue to write into DB.
* It's bring a fault tolrance now.

PROS

- Good for Write heavy application.
- Improves the Write Operation Latency. As writing into the DB happens asynchronously.
- Cache Hit chance increase a lot.
- Gives much better performance when used with read through Cache.

CONS :-

- If Data is removed from Cache and DB writes still not happens, then there is a chance of an issue. (It handled by keeping the TAT of cache little higher like 2 days).



Concept && Coding